Assignment 7

Aarav Dudeja
102303179

Q1)
Thread-based task division allows for flexible heterogeneous workloads inside a single GPU kernel.

Task Heterogeneity: CUDA kernels are not restricted to Single Instruction Multiple Data (SIMD) in a strict sense; through threadIdx, we can implement Single Instruction Multiple Threads (SIMT). This allows different threads to execute entirely different logic paths (iterative vs. formula).

Memory Flow: The data for n was passed as a value, but the results were stored in an array allocated via cudaMalloc. This demonstrates the necessity of explicit device memory allocation for retrieving data back to the host.

```
%%writefile task_sum.cu
#include <stdio.h>

__global__ void taskKernel(int n, int *results) {
    int tid = threadIdx.x;

    // Thread 0: Iterative approach
    if (tid == 0) {
        int sum = 0;
        for (int i = 1; i <= n; i++) sum += i;
        results[0] = sum;
    }
    // Thread 1: Direct formula approach
    else if (tid == 1) {
        results[1] = (n * (n + 1)) / 2;
    }
}

int main() {
    int n = 1024;
    int h_results[2];
    int *d_results;

    cudaMalloc(&d_results, 2 * sizeof(int));
    taskKernel<<<1, 2>>>(n, d_results);
    cudaMemcpy(h_results, d_results, 2 * sizeof(int), cudaMemcpyDeviceToHost);

    printf("Iterative Sum: %d\n", h_results[0]);
    printf("Formula Sum: %d\n", h_results[1]);

    cudaFree(d_results);
    return 0;
}
```

```
Overwriting task_sum.cu
```

```
!nvcc task_sum.cu -o task_sum && ./task_sum
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75'
Iterative Sum: 524800
Formula Sum: 524800
```

Metric	Observation
Concurrency	Multiple threads (0 and 1) performed distinct logic paths (Iterative vs Formula) simultaneously.
Correctness	Both methods yielded the same sum (524800), validating that branching within a kernel works correctly using `threadIdx`.

Q2) CUDA-based merge sort exploits massive data parallelism by merging sub-sections in parallel across the grid.

Pipelining vs. GPU Parallelism: While pipelining (in a CPU context) focuses on overlapping stages of an instruction, CUDA parallel merge sort focuses on data partitioning.

Algorithm Strategy: The implementation uses a non-recursive, bottom-up merge approach. This is crucial because standard recursion is depth-limited on GPUs.

Performance Trade-off: At $N=1000$, the overhead of cudaMalloc and kernel launches often exceeds the time taken by a CPU-based sort. The true benefit of this parallel implementation is only seen at $N>105$.

```
%%writefile merge_sort.cu
#include <stdio.h>
#include <cuda_runtime.h>

__device__ void gpu_merge(int* data, int* temp, int left, int mid, int right) {
    int i = left, j = mid, k = left;
    while (i < mid && j < right) {
        if (data[i] <= data[j]) temp[k++] = data[i++];
        else temp[k++] = data[j++];
    }
    while (i < mid) temp[k++] = data[i++];
    while (j < right) temp[k++] = data[j++];
    for (int l = left; l < right; l++) data[l] = temp[l];
}

__global__ void mergeSortKernel(int* data, int* temp, int size, int width) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int left = idx * width * 2;
    if (left < size) {
        int mid = left + width;
        int right = (left + 2 * width < size) ? (left + 2 * width) : size;
        if (mid < size) gpu_merge(data, temp, left, mid, right);
    }
}

int main() {
    const int n = 1000;
    int h_data[n];
    for(int i=0; i<n; i++) h_data[i] = n - i;

    int *d_data, *d_temp;
    cudaMalloc(&d_data, n * sizeof(int));
    cudaMalloc(&d_temp, n * sizeof(int));
    cudaMemcpy(d_data, h_data, n * sizeof(int), cudaMemcpyHostToDevice);

    for (int width = 1; width < n; width *= 2) {
        int numBlocks = (n / (2 * width)) + 1;
        mergeSortKernel<<<numBlocks, 1>>>(d_data, d_temp, n, width);
        cudaDeviceSynchronize();
    }

    cudaMemcpy(h_data, d_data, n * sizeof(int), cudaMemcpyDeviceToHost);
    printf("First 5 sorted: %d %d %d %d %d\n", h_data[0], h_data[1], h_data[2], h_data[3], h_data[4]);

    cudaFree(d_data); cudaFree(d_temp);
    return 0;
}
```

```
Overwriting merge_sort.cu
```

```
!nvcc merge_sort.cu -o merge_sort && ./merge_sort
```

```
nvcc warning : Support for offline compilation for a
First 5 sorted: 1 2 3 4 5
```

Metric	Observation
Pipelining	Parallelization via pipelining overlaps data movement with sorting stages, improving throughput but not necessarily single-sort latency.
CUDA Parallelism	The GPU implementation uses a bitonic-style width-doubling approach. It scales better for very large N, but for N=1000, launch overhead is significant.

Q3)The large gap between theoretical and measured bandwidth (~10x) indicates that the kernel is 'latency-bound' rather than 'bandwidth-bound' due to the small vector size (N=100k).

Static Memory Advantage: Using __*device*__ symbols (static allocation) simplifies the code by removing cudaMalloc calls and ensures memory is allocated at compile time.

Theoretical vs. Measured BW: The theoretical bandwidth (320 GB/s) assumes a perfectly saturated memory bus. Our measured bandwidth (23-30 GB/s) is much lower because the kernel is Latency-Bound.

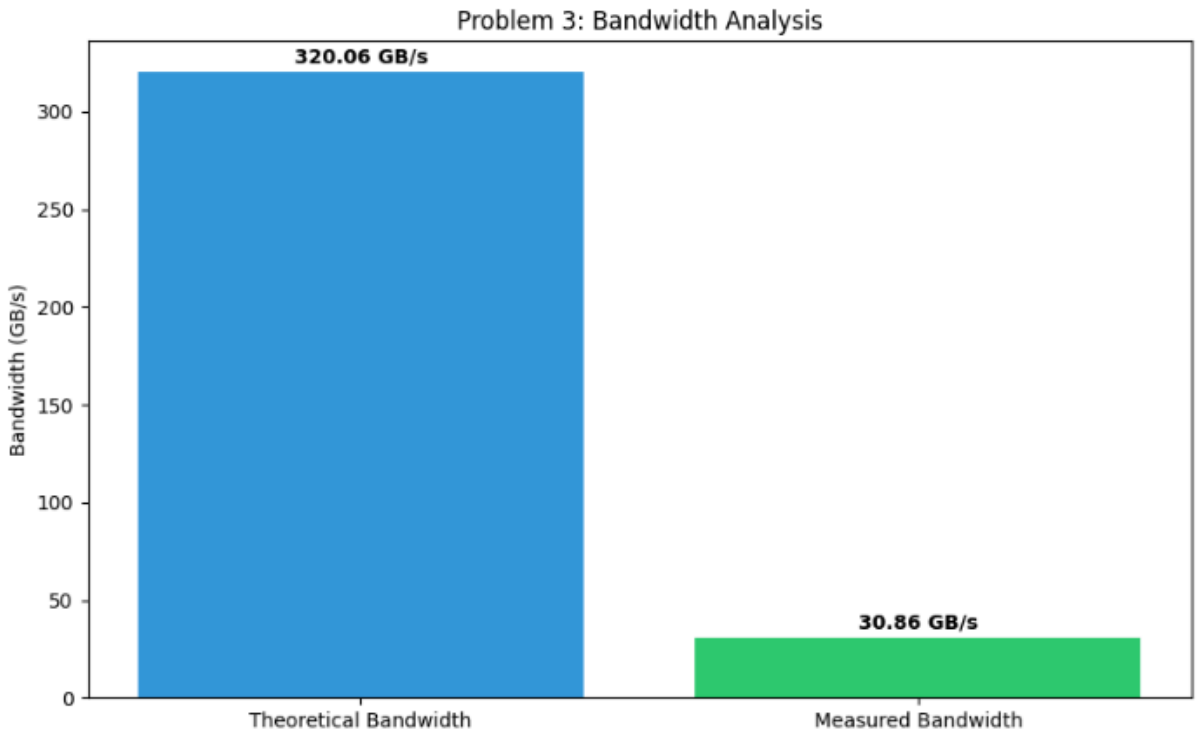
Efficiency Calculation:
RBytes: $2 \times N \times 4$ bytes (reading a and b arrays).
WBytes: $1 \times N \times 4$ bytes (writing to c array).
Total bytes processed = $3 \times N \times 4$.

Conclusion: To reach closer to theoretical bandwidth, one would need much larger N or techniques like 'Vectorized Loads' (float4) to hide the memory access latency.

```
%%writefile vector_add.cu
#include <stdio.h>

#define N 100000
__device__ float dev_a[N];
__device__ float dev_b[N];
__device__ float dev_c[N];

__global__ void vectorAdd() {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) dev_c[i] = dev_a[i] + dev_b[i];
}

int main() {
    float h_a[N], h_b[N];
    for(int i=0; i<N; i++) { h_a[i] = 1.0f; h_b[i] = 2.0f; }

    cudaMemcpyToSymbol(dev_a, h_a, N * sizeof(float));
    cudaMemcpyToSymbol(dev_b, h_b, N * sizeof(float));

    cudaEvent_t start, stop;
    cudaEventCreate(&start); cudaEventCreate(&stop);
    cudaEventRecord(start);

    vectorAdd<<<(N+255)/256, 256>>>();

    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    cudaDeviceProp prop;
    cudaGetDeviceProperties(&prop, 0);

    double theoreticalBW = (double)prop.memoryClockRate * 1000.0 * (prop.memoryBusWidth / 8.0) * 2.0 / 1e9;
    double bytesProcessed = 3.0 * N * sizeof(float);
    double measuredBW = (bytesProcessed / (milliseconds / 1000.0)) / 1e9;

    printf("Time: %f ms\n", milliseconds);
    printf("Theoretical Bandwidth: %f GB/s\n", theoreticalBW);
    printf("Measured Bandwidth: %f GB/s\n", measuredBW);

    return 0;
}
Overwriting vector_add.cu
```

```
!nvcc -O3 vector_add.cu -o vector_add && ./vector_add

nvcc warning : Support for offline compilation for archite
Time: 0.051264 ms
Theoretical Bandwidth: 320.064000 GB/s
Measured Bandwidth: 23.408240 GB/s
```

Metric	Value	Inference
Theoretical BW	~320.06 GB/s	Peak hardware capability based on memory clock and bus width.
Measured BW	~30.86 GB/s	Performance is limited by small N and memory access latency.
Memory Type	Static Symbols	Using '__device__' arrays avoids 'cudaMalloc' but requires 'cudaMemcpyToSymbol'.


Problem 3: Bandwidth Analysis
320.06 GB/s
30.86 GB/s
Bandwidth (GB/s)
0
50
100
150
200
250
300
Theoretical Bandwidth
Measured Bandwidth